LAPORAN PRAKTIKUM
OPERASI BLOK MEMORI

LAPORAN
Disusun untuk memenuhi tugas praktikum Mikroprosesor dan Interface

oleh :

Silfiana Dwi Nabiilah
221344028

Tanggal Percobaan: Jumat, 05 April 2024
Tanggal Penyerahan: Jumat, 19 April 2024

Instruktur: 1. Ferry Satria, BSEE., M.T.
2. Rahmawati Hasanah, S.ST., M.T.

PROGRAM STUDI D4 TEKNIK TELEKOMUNIKASI
JURUSAN TEKNIK ELEKTRO
POLITEKNIK NEGERI BANDUNG
2024

DAFTAR ISI

DAFTAR ISI 2
A. TUJUAN 3
B. LANDASAN TEORI 3
C. ALAT DAN BAHAN 4
D. LANGKAH KERJA 4
E. SOAL DAN HASIL 5
F. ANALISA DATA 15
G. KESIMPULAN 16
H. SARAN 16
DAFTAR PUSTAKA 17

A. TUJUAN

1. Memenuhi tugas mata kuliah Mikroprosesor dan Interface.
2. Memahami prinsip kerja dari Aplikasi simulator EMU 8086.
3. Membuat program sederhana menggunakan Bahasa Assembly.
4. Menganalisa hasil program yang dibuat dengan output yang dihasilkan mengenai program blok memori.

B. LANDASAN TEORI

1. Pengertian Mikroprosesor

Basis Data adalah sekumpulan informasi (data) yang berada di dalam komputer sehingga bisa diperiksa menggunakan sistem. Komponen dasar ini dijadikan sebagai sebuah sistem yang dapat digunakan untuk memudahkan pengguna dalam pengelolaan informasi yang dibutuhkan. Pada dunia pemrograman basis data sering kali digunakan untuk menyimpan konfigurasi, data pemograman, dan log hasil dari proses *running* aplikasi program.

Mikroprosesor merupakan CPU (Central Processing Unit) yang dikemas dalam satu buah IC (Integrated Circuit) atau chip. Mikroprosesor memiliki kemampuan untuk menerima input data digital kemudian diproses sesuai intruksi dalam memori untuk menghasilkan output. Mikroprosesor beroperasi dalam sistem angka biner dengan logika kombinasional dan logika digital sekuensial. Didalamnya terdapat

ALU (unit aritmatika dan logika), register, control unit, bus, dan clock untuk komputasi. Mikroprosesor mampu menjalankan perintah secara berurutan dalam bentuk program dengan intruksi yang menggunakan sistem bilangan biner. Akan tetapi, tidak semua mikroprosesor kompatibel untuk menjalankan berbagai program dalam kode intruksi yang berbeda. Beberapa jenis mikroprosesor seperti Z80 hanya dapat menjalankan intruksi untuk mikroprosesor 8080 atau 8085, sedangkan program yang menggunakan prosesor 8086 hanya dapat dijalankan dengan prosesor yang lebih tinggi seperti 80186, 80286, dan seterusnya. Sebaliknya, program 80186, 80286 dan seterusnya tidak berkemampuan untuk dijalankan dengan prosesor dibawahnya.

2. Pengertian EMU8086

Emu8086 adalah emulator mikroprosesor 8086 yang dapat digunakan untuk mengemulasikan perangkat keras pada komputer Windows. Emulator ini memungkinkan pengguna untuk menjalankan program yang dibuat untuk mikroprosesor 8086 pada komputer modern.

Emu8086 juga dilengkapi dengan assembler terintegrasi dan tutorial untuk pemula. Emulator ini dapat digunakan untuk mengeksplorasi dan mempelajari bahasa pemrograman assembly serta untuk mengembangkan dan menguji kode assembly pada mikroprosesor 8086.

3. Bahasa Assembly

Bahasa Assembly (rakitan) adalah bahasa pemrograman tingkat rendah yang digunakan pada perangkat yang dapat di program seperti komputer dan mikroprosesor. Instruksi-instruksi pada bahasa assembly menggantikan kode-kode biner dari bahasa mesin dengan *mnemonic* yang mudah diingat. Misal, instruksi penambahan dalam bahasa mesin memiliki kode “10110011” sedangkan dalam bahasa assembly dapat dibuat dalam instruksi mnemonic ADD, sehingga lebih mudah diapahami oleh manusia.

Bahasa assembly mempunyai hubungan 1-1 dengan berdasarkan instruksi mesin. Contohnya, untuk setiap julukan atau *mnemonic* yang nanti akan ditulis dengan menggunakan bahasa assembly, maka nanti juga akan langsung diterjemahkan dengan tepat satu kode operasi sehingga bisa langsung dimengerti oleh komputer. Untuk memahami cara kerja bahasa Assembly.

C. ALAT DAN BAHAN

1. Laptop atau PC
2. Simulator EMU 8086

D. LANGKAH KERJA

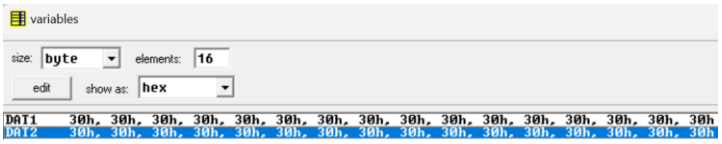
1. Mendownload aplikasi simulator EMU 8086 pada laptop atau PC.
2. Instalisasi aplikasi EMU 8086 pada laptop atau PC.
3. Memulai untuk membuat New Project pada aplikasi Emu 8086.
4. Memulai membuat pemrograman menggunakan Bahasa Assembly.
5. Menguji hasil pemrograman dengen mencompile pada aplikasi Emu8086.

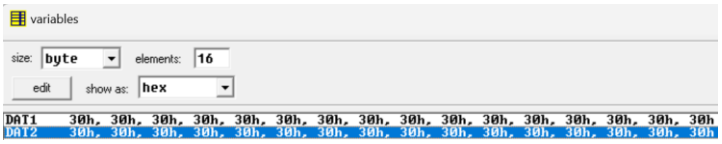
E. SOAL, HASIL DAN ANALISA

1) Buat diagram alir dan program untuk memindahkan (meng-copy) 16 data 8 bit dari Blok Memori 1 ke Blok Memori 2. Tentukan sendiri data yang dipergunakan. (Gunakan 2 Cara).

Jawab:

Program	Hasil Output

`TITLE LAB7 NO 1 (CARA 1)`
`.MODEL SMALL`
`.STACK 100H`
 	
`.DATA`
`DAT1 DB 16 DUP(30H)`
`DAT2 DB 16 DUP(0)`
 	
`.CODE`
`MAIN PROC`
`MOV AX,@DATA`
`MOV DS,AX`
`MOV ES,AX`
 	
`MOV CX,16`
 	
`MOV SI,OFFSET DAT1 ; mengatur sumber ke DAT1`
`MOV DI,OFFSET DAT2 ; mengatur destinasi ke DAT2`
 	
`CLD ;Set Direction Flag (arah mundur)`
`REP MOVSB`
 	
`MOV AH,4CH`
`INT 21H`
 	
`MAIN ENDP`
`END MAIN`
	

`TITLE LAB7 NO 1 (CARA 2)`
`.MODEL SMALL`
`.STACK 100H`
 	
`.DATA`
`DAT1 DB 16 DUP(30H)`
`DAT2 DB 16 DUP(0)`
 	
`.CODE`
`MAIN PROC`
`MOV AX,@DATA`
`MOV DS,AX`
`MOV ES,AX`
 	
`MOV CX,16`
`MOV SI, OFFSET DAT1 ; mengatur sumber ke DAT1`
`MOV DI, OFFSET DAT2 ; mengatur destinasi ke DAT2`
 	
`CLD ; Set Direction Flag (arah mundur)`
 	
`L1:`
`LODSB ; Load byte dari [SI] ke AL`
`STOSB ; Store byte dari AL ke [DI], dan update SI dan DI`
`LOOP L1 ; Ulangi sampai CX = 0`
 	
`MOV AH,4CH`
`INT 21H`
 	
`MAIN ENDP`
`END MAIN`
	

Analisa:

Pada program nomor 1 cara pertama ini berfungsi untuk menyalin atau memindahkan 16 data 8 bit dari satu blok memori menuju blok memori lain, pada cara ini mengatur terlebih dahulu counter CX menjadi 16 (karena pada soal ini memiliki 16 byte data yang akan disalin), lalu selanjutnya mengatur SI Ke offset dari DAT1 dan mengatur DI ke offset dari DAT2 serta mengatur direction flag (DF) menjadi 0 yang menandakan arah gerakan string mundur dengan menuliskan program CLD, setelah selesai mengatur beberapa program maka selanjutnya program akan mengulangi instruksi MOVSB sebanyak CX kali yang menggerakan byte dari alamat SI menuju alamat DI. Dikarenakan DF telah diatur ke 0 (mundur), maka hal ini yang akan menyebabkan penggandaan dari DAT1 dan DAT2 dengan arah mundur. Maka dari itu, nantinya program ini akan menyalin data dari DAT1 ke DAT2 sebanyak 16 byte dengan arah mundur.

Jika cara pertama instruksi yang digunakan untuk menggandakan data adalah REP MOVSB Instruksi ini secara otomatis menggerakkan data dari lokasi sumber (SI) ke lokasi tujuan (DI) sebanyak CX kali. Dengan mengatur direction flag (DF) menjadi 0 (CLD), gerakan string akan dilakukan mundur. Sementara itu, pada program kedua (Cara 2), instruksi yang digunakan untuk menggandakan data adalah LODSB dan STOSB dalam loop. Instruksi LODSB memuat byte dari alamat SI ke dalam register AL, sedangkan instruksi STOSB menyimpan byte dari AL ke alamat DI, dan mengupdate SI dan DI. Loop digunakan untuk mengulangi proses ini sebanyak CX kali.

Secara keseluruhan, keduanya mencapai tujuan yang sama, yaitu menyalin data dari DAT1 ke DAT2. Namun, metode pelaksanaan sedikit berbeda antara keduanya. Program pertama menggunakan instruksi REP MOVSB untuk gerakan string, sementara program kedua menggunakan instruksi LODSB dan STOSB dalam loop.

2) Buat diagram alir dan program untuk memindahkan (meng-copy) 16 data 16 bit dari Blok Memori 1 ke Blok Memori 2. Tentukan sendiri data yang dipergunakan. (Gunakan 2 Cara).

Jawab:

Program dan Hasil Output

```
TITLE LAB7 NO 2 (CARA 1)
.MODEL SMALL
.STACK 100H

.DATA
DAT1 DW 16 DUP(0A123H)
DAT2 DW 16 DUP(0)

.CODE
MAIN PROC
     MOV AX,@DATA
     MOV DS,AX
     MOV ES,AX

     MOV CX,16          ; Memuat nilai 16 ke dalam register CX (jumlah elemen yang akan dipindahkan)

     STD                ; Mengatur arah gerakan string menjadi mundur
     MOV DI,OFFSET DAT1 ; Mengatur destinasi awal (DI) ke alamat offset variabel DAT1
     MOV DI,OFFSET DAT2 ; Mengatur destinasi awal (DI) lagi ke alamat offset variabel DAT2,
                        ; yang menggantikan alamat DI yang telah ditetapkan sebelumnya

     CLD                ; Mengatur arah gerakan string menjadi maju
     REP MOVSW          ; Mengulang instruksi MOVSW sebanyak CX kali,
                        ; yang memindahkan word dari DS:SI ke ES:DI sebanyak 16 kali

     MOV AH,4CH
     INT 21H

MAIN ENDP
END MAIN
```

variables

size: word elements: 16

edit show as: hex

DAT1 0A123h, 0A123h, 0A123h, 0A123h, 0A123h, 0A123h, 0A123h, 0A123h, 0A123h, 0A123h, 0A123h, 0A123h, 0A123h, 0A123h, 0A123h, 0A123h

DAT2 0A123h, 0A123h, 0A123h, 0A123h, 0A123h, 0A123h, 0A123h, 0A123h, 0A123h, 0A123h, 0A123h, 0A123h, 0A123h, 0A123h, 0A123h, 0A123h

```
TITLE LAB7 NO2 (CARA 2)
.MODEL SMALL
.STACK 100H

.DATA
DAT1 DW 16 DUP(0B456H)
DAT2 DW 16 DUP(0)

.CODE
MAIN PROC
     MOV AX,@DATA
     MOV DS,AX
     MOV ES,AX

     MOV CX,16    ; Memuat nilai 16 ke dalan register CX (jumlah elemen yang akan dipindahkan)
     MOV DI, OFFSET DAT1 ; mengatur sumber ke DAT1
     MOV DI, OFFSET DAT2 ; mengatur destinasi ke DAT2

     CLD ; Set Direction Flag (arah mundur)

     L1:
     LODSW ; Load byte dari [SI] ke AL
     STOSW ; Store byte dari AL ke [DI], dan update SI dan DI
     LOOP L1 ; Ulangi sampai CX = 0

     MOV AH,4CH
     INT 21H

MAIN ENDP
END MAIN
```

variables

size: word elements: 16

edit show as: hex

DAT1 0B456h, 0B456h, 0B456h, 0B456h, 0B456h, 0B456h, 0B456h, 0B456h, 0B456h, 0B456h, 0B456h, 0B456h, 0B456h, 0B456h, 0B456h, 0B456h

DAT2 0B456h, 0B456h, 0B456h, 0B456h, 0B456h, 0B456h, 0B456h, 0B456h, 0B456h, 0B456h, 0B456h, 0B456h, 0B456h, 0B456h, 0B456h, 0B456h

Analisa:

Pada program nomor 2 cara pertama ini berfungsi untuk menyalin atau memindahkan 16 data 16 bit dari satu blok memori menuju blok memori lain, pada cara ini dimulai dengan mengatur arah gerakan string menjadi mundur menggunakan perintah STD, lalu selanjutnya mengatur DI Ke offset dari DAT1 dan mengatur DI ke offset dari DAT2 yang menggantikan alamat DI yang telah ditetapkan sebelumnya, serta mengatur arah gerakan string menjadi maju dengan perintah CLD, selanjutnya mengulangi instruksi MOVSW sebanyak CX kali, yang memindahkan word (16-bit) dari DS:SI ke ES:DI sebanyak 16 kali dengan perintah REP MOVSW. Maka dari itu, nantinya program ini akan memindahkan data dari DAT1 ke DAT2 sebanyak 16 elemen dengan arah mundur.

Jika cara oertama menggunakan intruksi REP MOVSW yang secara otomatis menggerakkan word dari lokasi sumber (DS:SI) ke lokasi tujuan (ES:DI) sebanyak CX kali. Dengan mengatur direction flag (DF) menjadi 1 (STD), gerakan string akan dilakukan mundur. Sementara itu, pada cara kedua instruksi yang digunakan adalah LODSW dan STOSW dalam loop. Instruksi LODSW memuat word dari alamat SI ke dalam register AX, sedangkan instruksi STOSW menyimpan word dari AX ke alamat DI, dan mengupdate SI dan DI. Loop digunakan untuk mengulangi proses ini sebanyak CX kali.

Secara keseluruhan, keduanya mencapai tujuan yang sama, yaitu memindahkan data dari DAT1 ke DAT2. Namun, metode pelaksanaan sedikit berbeda antara keduanya. Program pertama menggunakan instruksi REP MOVSW untuk gerakan string, sementara program kedua menggunakan instruksi LODSW dan STOSW dalam loop.

3) Buat diagram alir dan program untuk mengisi 16 data Heksadesimal 8 bit (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F) pada suatu Blok Memori.

Jawab:

Program dan Hasil Output

```
TITLE LAB07 NO3
.MODEL SMALL
.STACK 100H

.DATA
DAT1 DB 16 DUP(0)

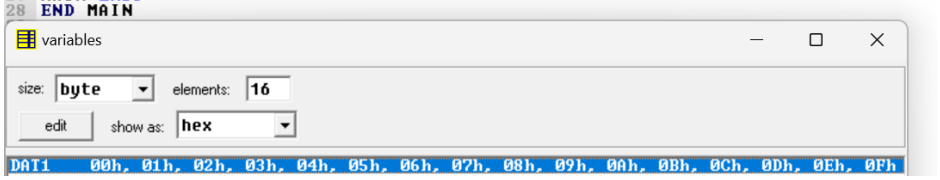
.CODE
MAIN PROC
     MOV AX,@DATA
     MOV DS,AX
     MOV ES,AX

     MOV CX,16
     CLD
     LEA DI,DAT1     ; Mengatur DI ke alamat offset dari variabel DAT1
     MOV AL,0

L1:  MOV [DI], AL    ; Menyimpan nilai AL di alamat yang ditunjuk oleh DI
     INC AL          ; Menaikkan nilai AL untuk iterasi berikutnya
     INC DI          ; Menaikkan DI untuk menunjuk ke lokasi memori berikutnya
     LOOP L1         ; Mengulangi sampai CX = 0

     MOV AH,4CH
     INT 21H

MAIN ENDP
END MAIN
```



Analisa:

Pada program nomor 3 ini berfungsi untuk mengisi 16 data Heksadesimal 8 bit (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F) pada suatu Blok Memori dengan cara mengatur counter CX menjadi 16, karena kita memiliki 16 elemen yang akan diisi, selanjutnya mengatur arah gerakan string menjadi maju (DF = 0) dengan perintah CLD, lalu mengatur Destinasi Indeks (DI) ke alamat offset dari variabel DAT1 dengan perintah LEA DI,DAT1 dan dilanjut dengan membuat nilai AL menjadi 0.

Pada program ini digunakan loop dengan label L1, perintah awal yang digunakan adalah MOV [DI],AL yang berfungsi untuk Menyimpan nilai AL di alamat yang ditunjuk oleh DI (yaitu, mengisi elemen array DAT1), selanjutnya perintah INC AL yang berfungsi menaikkan nilai AL untuk iterasi berikutnya dan INC DI yang berfungsi menaikkan DI untuk menunjuk ke lokasi memori berikutnya, perogram loop ini ditutup dengan perintah LOOP L1 yang berfungsi untuk mengulangi proses sampai CX = 0, yaitu sebanyak 16 kali.

Dengan program tersebut maka nantinya array pada DAT1 akan diisi dengan urutan nilai dari 0 shingga nilai 15.

4) Buat diagram alir dan program untuk mengisi 16 data Heksadesimal 16 bit (0, 1111H, 2222H, 3333H, 4444H, 5555H, 6666H, 7777H, 8888H, 9999H, AAAAH, BBBBH, CCCCH, DDDDH, EEEEH, FFFFH) pada suatu Blok Memori.

Jawab:

Program dan Hasil Output

```
TITLE LAB07 NO4
.MODEL SMALL
.STACK 100H

.DATA
DAT1 DW 32 DUP(0)

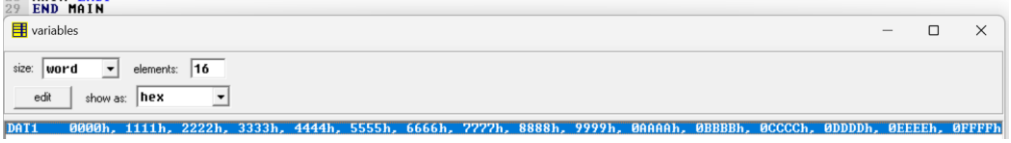
.CODE
MAIN PROC
     MOV AX,@DATA
     MOV DS,AX
     MOV ES,AX

     MOV CX,32     ; Memuat nilai 32 ke dalam register CX (jumlah iterasi yang akan dilakukan)
     CLD           ; Mengatur arah gerakan string menjadi maju (clear direction flag)
     LEA DI,DAT1   ; Mengatur DI ke alamat offset dari variabel DAT1
     MOV AX,0      ; Memuat nilai 0 ke dalam register AX (nilai awal untuk array DAT1)

L1:                ; Label untuk awal loop
     STOSW         ; Menyimpan nilai yang ada di dalam register AX ke alamat yang ditunjuk oleh
                   ;DI (array DAT1) dan menambahkan 2 ke DI
     ADD AX,1111H  ; Menambahkan 1111H ke dalam register AX untuk menghasilkan nilai selanjutnya
     LOOP L1       ; Mengulangi loop selama CX belum mencapai 0 (CX - 1 != 0)

     MOV AH,4CH
     INT 21H

MAIN ENDP
END MAIN
```



Analisa:

Pada program nomor 4 ini sebetulnya hampir sama dengan program nomor 3, yang membedakannya adalah nilai-nilai di dalam array DAT1 dihasilkan secara iteratif dengan menggunakan instruksi STOSW dan ADD. Nilai awal 0 dimasukkan ke dalam register AX, kemudian disimpan di alamat yang ditunjuk oleh DI (array DAT1) dengan instruksi STOSW. Setelah itu, nilai di register AX ditambahkan dengan 1111H untuk menghasilkan nilai selanjutnya yang akan disimpan di array DAT1. Proses ini diulangi sebanyak 32 kali.

Pada program nomor 4 ini, STOSW digunakan untuk menyimpan nilai yang ada di dalam register AX (yang awalnya diisi dengan 0) ke alamat yang ditunjuk oleh DI (yang awalnya diatur ke alamat offset dari variabel DAT1). Setelah nilai disimpan, register DI akan otomatis ditambahkan dengan 2, karena STOSW mengoperasikan word (16-bit), sehingga DI akan menunjuk ke alamat berikutnya dalam array DAT1. Sedangkan instruksi ADD pada program ini, digunakan untuk menambahkan nilai 1111H ke dalam register AX setiap kali iterasi dilakukan. Dengan demikian, nilai yang disimpan di dalam array DAT1 akan bertambah sebesar 1111H setiap kali iterasi dilakukan.

Secara keseluruhan, keduanya mencapai tujuan yang sama, yaitu mengisi array DAT1 dengan nilai-nilai tertentu. Namun, metode pengisian datanya sedikit

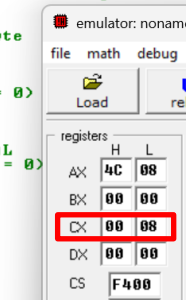
berbeda antara keduanya. Program nomor 3 menggunakan instruksi MOV dan INC dalam perulangan, sementara program nomor 4 menggunakan instruksi STOSW dan ADD dalam perulangan.

5) Buat diagram alir dan program untuk mencari nilai data terkecil dari 8 data biner tidak bertanda 8 bit: 23H, 56H, 22H, 8AH, F3H, 66H, 8H, 7CH. Tempatkan nilai data terkecil tersebut pada register CL.

Jawab:

Program dan Hasil Output

```
01 TITLE LAB07 NO5
02 .MODEL SMALL
03 .STACK 100H
04
05 .DATA
06 DAT1 DB 23H,56H,22H,8AH,0F3H,66H,8H,7CH
07
08 .CODE
09 MAIN PROC
10      MOV AX,@DATA
11      MOV DS,AX
12      MOV ES,AX
13
14      MOV CX,8          ; Memuat nilai 8 ke dalam register CX (jumlah elemen yang akan diproses)
15      MOV AL,DAT1       ; Memuat byte pertama dari DAT1 ke dalam register AL
16      LEA DI,DAT1       ; Mengatur DI ke alamat offset dari variabel DAT1
17      CLD               ; Mengatur arah gerakan string menjadi maju (clear direction flag)
18
19 L1:                   ; Label untuk awal loop
20      SCASB             ; Membandingkan byte yang ada di alamat ES:DI dengan byte
21                        ; yang ada di dalam register AL
22      JA L2             ; Melompat ke L2 jika byte di alamat ES:DI lebih besar
23                        ; daripada byte yang ada di dalam register AL
24      LOOP L1           ; Mengulangi loop selama CX belum mencapai 0 (CX - 1 != 0)
25      JMP L3            ; Melompat ke L3
26
27 L2:                   ; Label L2
28      MOV AL,DAT[DI-1] ; Memuat byte sebelumnya dari DAT1 ke dalam register AL
29      LOOP L1           ; Mengulangi loop selama CX belum mencapai 0 (CX - 1 != 0)
30
31 L3:                   ; Label L3
32      MOV CL,AL         ; Memuat nilai dari register AL ke dalam register CL
33
34      MOV AH,4CH
35      INT 21H
36
37 MAIN ENDP
38 END MAIN
```



Analisa:

Pada program nomor 5 ini berfungsi untuk mencari nilai data terkecil dari 8 data biner tidak bertanda 8 bit: 23H, 56H, 22H, 8AH, F3H, 66H, 8H, 7CH dan menempatkan nilai data terkecil tersebut pada register CL setelah mengatur register dan arah gerak menjadi maju (DF = 0), maka pada program ini digunakan loop L1, L2, dan L3. Pada Loop L1 ini berfungsi untuk membandingkan byte yang ada di alamat ES:DI dengan byte yang ada di dalam register AL dengan instruksi SCASB dan melakukan lompatan ke L2 jika byte di alamat ES:DI lebih besar daripada byte yang ada di dalam register AL menggunakan perintah JA L2, serta yang terakhir yaitu m engulangi loop selama CX belum mencapai 0 (CX - 1 != 0) dengan perintah LOOP L1 dan menutupnya dengan melompat ke L3 jika loop selesai menggunakan perintah JMP L3.

Sedangkan pada loop L2 berfungsi m emuat byte sebelumnya dari array DAT1 ke dalam register AL dengan perintah MOV AL,DAT[DI-1] serta mengulangi loop selama CX belum mencapai 0 (CX - 1 != 0) dengan perintah

LOOP L1. Dan yang terakhir pada loop L3 berfungsi untuk Memuat nilai dari register AL ke dalam register CL. Pada titik ini, nilai terkecil sudah ditemukan dan disimpan di register CL dengan perintah MOV CL,AL.

Dengan demikian, setelah program dieksekusi, nilai terkecil dari array DAT1 akan disimpan di dalam register CL.

6) Buat diagram alir dan program untuk mencari nilai data terkecil dari 8 data biner bertanda 8 bit: 23H, 56H, 22H, 8AH, F3H, 66H, 8H, 7CH. Tempatkan nilai data terkecil tersebut pada register CL.

Jawab:

Program dan Hasil Output

```
TITLE LAB07 NO6
.MODEL SMALL
.STACK 100H

.DATA
DAT1 DB 23H,56H,22H,8AH,0F3H,66H,8H,7CH

.CODE
MAIN PROC
     MOV AX,@DATA
     MOV DS,AX
     MOV ES,AX

     MOV CX,8          ; Memuat nilai 8 ke dalam register CX (jumlah elemen yang akan diproses)
     MOV AL,DAT1       ; Memuat byte pertama dari DAT1 ke dalam register AL
     LEA DI,DAT1       ; Mengatur DI ke alamat offset dari variabel DAT1
     CLD               ; Mengatur arah gerakan string menjadi maju (clear direction flag)

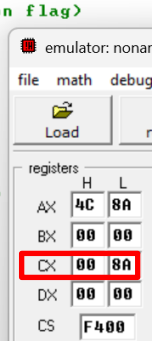
L1:                    ; Label untuk awal loop
     SCASB             ; Membandingkan byte yang ada di alamat ES:DI dengan byte
                       ; yang ada di dalam register AL
     JG L2             ; Melompat ke L2 jika nilai AL lebih besar dari 0
                       ; daripada byte yang ada di dalam register AL
     LOOP L1           ; Mengulangi loop selama CX belum mencapai 0 (CX - 1 != 0)
     JMP L3            ; Melompat ke L3

L2:                    ; Label L2
     MOV AL,DAT[DI-1] ; Memuat byte sebelumnya dari DAT1 ke dalam register AL
     LOOP L1           ; Mengulangi loop selama CX belum mencapai 0 (CX - 1 != 0)

L3:                    ; Label L3
     MOV CL,AL         ; Memuat nilai dari register AL ke dalam register CL

     MOV AH,4CH
     INT 21H

MAIN ENDP
END MAIN
```



Analisa:

Pada dasarnya, program nomor 6 ini mirip dengan program nomo 5 yang membedakannya yaitu pada program ini perbandingan dilakukan menggunakan instruksi JG (Jump if Greater), yang berarti melompat jika nilai AL lebih besar dari nilai byte yang dibandingkan.

Karena instruksi perbandingan berbeda, langkah-langkah yang diambil juga berbeda: Pada program nomor 5, jika byte di alamat ES:DI lebih besar dari nilai yang dibandingkan (AL), maka program akan langsung melompat ke label L2 dan mengulangi loop. Sedangkan pada program nomor 6, jika nilai AL lebih kecil dari byte yang dibandingkan, program akan melanjutkan iterasi loop, dan jika tidak, program akan melompat ke label L3 setelah loop selesai.

7) Buat diagram alir dan program untuk mencari nilai data terkecil dari 8 data biner tidak bertanda 16 bit. Tentukan sendiri data uji yang diperlukan. Tempatkan nilai data terkecil tersebut pada register DX.

Jawab:

Program dan Hasil Output

```
TITLE LAB07 NO7
.MODEL SMALL
.STACK 100H

.DATA
DAT1 DW 1212H,2323H,3434H,5656H,6767H,7878H,8989,0101H

.CODE
MAIN PROC
     MOV AX,@DATA
     MOV DS,AX
     MOV ES,AX

     MOV CX,16          ; Memuat nilai 16 ke dalam register CX (jumlah elemen yang akan diproses)
     MOV AX,DAT1        ; Memuat nilai pertama dari DAT1 ke dalam register AX
     LEA DI,DAT1        ; Mengatur DI ke alamat offset dari variabel DAT1
     CLD                ; Mengatur arah gerakan string menjadi maju (clear direction flag)

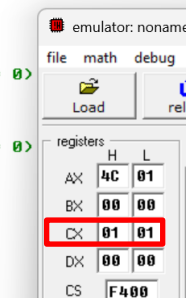
L1:                      ; Label untuk awal loop
     SCASB               ; Membandingkan byte yang ada di alamat ES:DI dengan
                         ; byte yang ada di dalam register AL
     JA L2               ; Melompat ke L2 jika nilai AL lebih besar dari byte
                         ; yang ada di dalam register AL
     LOOP L1             ; Mengulangi loop selama CX belum mencapai 0 (CX - 1 != 0)
     JMP L3              ; Melompat ke L3

L2:                      ; Label L2
     MOV AX,DAT[DI-2]    ; Memuat byte sebelumnya dari DAT1 ke dalam register AX
     LOOP L1             ; Mengulangi loop selama CX belum mencapai 0 (CX - 1 != 0)

L3:                      ; Label L3
     MOV CX,AX           ; Memuat nilai dari register AX ke dalam register CX


     MOV AH,4CH
     INT 21H

MAIN ENDP
END MAIN
```



Analisa:

Pada program nomor 7 ini sebetulnya hampir sama konsepnya dengan program nomor 5, jika pada program nomor 5 bernilai 8 bit maka pada program nomor 7 ini bernilai 16 bit.

Pada program nomor 5 perbandingan yang digunakan adalah SCASB yang membandingkan byte yang ada di alamat ES:DI dengan byte yang ada di dalam register AL. Kemudian, jika byte di alamat tersebut kurang dari byte yang ada di dalam register AL, program akan melompat ke label L2 'JA L2. Sedangkan pada program nomor 7, instruksi perbandingan yang digunakan adalah JA (Jump if Above), yang melompat ke label L2 jika nilai di dalam register AL lebih besar dari byte yang ada di alamat ES:DI.

Selain itu, kedua program tersebut memiliki struktur yang serupa dalam langkah-langkah yang diambil untuk mencapai tujuan akhir, yaitu menemukan nilai terkecil dari array DAT1.

8) Buat diagram alir dan program untuk mencari nilai data terkecil dari 8 data biner bertanda 16 bit. Tentukan sendiri data uji yang diperlukan. Tempatkan nilai data terkecil tersebut pada register DX.

Jawab:

Program dan Hasil Output

```
TITLE LAB07 NO8
.MODEL SMALL
.STACK 100H

.DATA
DAT1 DW 2121H,1256H,2132H,138AH,0F31H,6116H,0218H,27CH

.CODE
MAIN PROC
     MOV AX,@DATA
     MOV DS,AX
     MOV ES,AX

     MOV CX,16           ; Memuat nilai 16 ke dalam register CX (jumlah elemen yang akan diproses)
     MOV AX,DAT1         ; Memuat nilai pertama dari DAT1 ke dalam register AX
     LEA DI,DAT1         ; Mengatur DI ke alamat offset dari variabel DAT1
     CLD                 ; Mengatur arah gerakan string menjadi maju (clear direction flag)

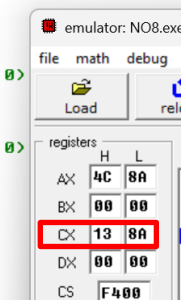
L1:                      ; Label untuk awal loop
     SCASB               ; Membandingkan byte yang ada di alamat ES:DI dengan
                         ; byte yang ada di dalam register AL
     JG L2               ; Melompat ke L2 jika nilai AL lebih besar dari 0
                         ; yang ada di dalam register AL
     LOOP L1             ; Mengulangi loop selama CX belum mencapai 0 (CX - 1 != 0)
     JMP L3              ; Melompat ke L3

L2:                      ; Label L2
     MOV AX,DAT[DI-2]    ; Memuat byte sebelumnya dari DAT1 ke dalam register AX
     LOOP L1             ; Mengulangi loop selama CX belum mencapai 0 (CX - 1 != 0)

L3:                      ; Label L3
     MOV CX,AX           ; Memuat nilai dari register AX ke dalam register CX


     MOV AH,4CH
     INT 21H

MAIN ENDP
END MAIN
```



Analisa:

Pada dasarnya program nomor 8 ini konsep nya sama dengan program nomor 7, hanya saja pada program nomor 7 ini instruksi JA digunakan setelah instruksi SCASB untuk memeriksa apakah byte di alamat tersebut lebih besar dari byte yang ada di dalam register AL. Sedengkan pada program nomor 8 perbedaannya terletak pada kondisi yang sama, yaitu instruksi JG digunakan setelah instruksi SCASB untuk memeriksa apakah byte di alamat tersebut lebih besar dari nilai 0.

Jadi, perbedaan utamanya terletak pada kondisi perbandingan dan cara penanganan kondisi tersebut dalam mencari nilai terkecil dari array DAT1.

9) Buat diagram alir dan program untuk mencari nilai data terkecil dari 8 data biner tidak bertanda 24 bit. Tentukan sendiri data uji yang diperlukan. Tempatkan nilai data terkecil tersebut pada register CL:DX.

Jawab:

Program dan Hasil Output

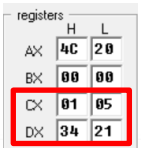
```
TITLE LAB07 NO9
.MODEL SMALL    ;Simpan data terkecil di:Reg CL:DX
.STACK 100H
.DATA
DAT DB 65H,8CH,36H  ; 368C65H
    DB 0FFH,45H,77H ; 7745FFH
    DB 54H,76H,98H  ; 987654H
    DB 0D4H,6CH,0A5H; A56CD4H
    DB 88H,77H,66H  ; 667788H
    DB 66H,45H,77H  ; 774566H
    DB 6FH,8CH,36H  ; 368C6FH
    DB 21H,34H,05H  ; 053421H
.CODE
MAIN    PROC
        MOV AX, @DATA
        MOV DS,AX
        MOV SI,0
        MOV DL,[DAT+SI]   ; Memuat byte pertama dari data ke dalam register DL
        MOV DH,[DAT+SI+1] ; Memuat byte kedua dari data ke dalam register DH
        MOV CL,[DAT+SI+2] ; Memuat byte ketiga dari data ke dalam register CL
        ADD SI,3        ; Menambahkan 3 ke SI untuk menggeser ke data berikutnya dalam array DAT

LUP1:                   ; Label untuk awal loop
        CMP CL,[DAT+SI+2]  ; Membandingkan byte ketiga dari data saat ini dengan byte ketiga dari data selanjutnya
        JA BSR              ; Melompat ke BSR jika byte ketiga dari data saat ini lebih besar dari byte ketiga dari data selanjutnya
        JB KCL              ; Melompat ke KCL jika byte ketiga dari data saat ini lebih kecil dari byte ketiga dari data selanjutnya
        CMP DH,[DAT+SI+1]  ; Membandingkan byte kedua dari data saat ini dengan byte kedua dari data selanjutnya
        JA BSR              ; Melompat ke BSR jika byte kedua dari data saat ini lebih besar dari byte kedua dari data selanjutnya
        JB KCL              ; Melompat ke KCL jika byte kedua dari data saat ini lebih kecil dari byte kedua dari data selanjutnya
        CMP DL,[DAT+SI]    ; Membandingkan byte pertama dari data saat ini dengan byte pertama dari data selanjutnya
        JA BSR              ; Melompat ke BSR jika byte pertama dari data saat ini lebih besar dari byte pertama dari data selanjutnya
        JB KCL              ; Melompat ke KCL jika byte pertama dari data saat ini lebih kecil dari byte pertama dari data selanjutnya
        JMP KCL             ; Melompat ke KCL jika semua byte sama

BSR:    MOV DL,[DAT+SI]   ; Menyalin byte pertama dari data saat ini ke dalam register DL
        MOV DH,[DAT+SI+1] ; Menyalin byte kedua dari data saat ini ke dalam register DH
        MOV CL,[DAT+SI+2] ; Menyalin byte ketiga dari data saat ini ke dalam register CL

KCL:    ADD SI,3        ; Menambahkan 3 ke SI untuk menggeser ke data berikutnya dalam array DAT
        CMP SI,24       ; Membandingkan SI dengan 24 (ukuran total data dalam bit)
        JNE LUP1        ; Melompat kembali ke LUP1 jika SI belum mencapai 24
        MOV AH,4CH
        INT 21H

MAIN ENDP
END MAIN
```



Analisa:

Program ini berfungsi untuk mencari nilai terkecil dari biner 24 bit tidak bertanda dan disimpan pada register CL:DX. Pada program ini data dipisah menjadi 8 bit 8 bit. Pembandingan yang dilakukan dimulai dari data paling kiri atau MSB. Jika data yang sudah ada pada register tujuan lebih besar maka akan terjadi pemindahan data yang jadi pembanding ke register tujuan.

10) Buat diagram alir dan program untuk mencari nilai data terkecil dari 8 data biner bertanda 24 bit. Tentukan sendiri data uji yang diperlukan. Tempatkan nilai data terkecil tersebut pada register CL:DX.

Jawab:

Program dan Hasil Output

```
TITLE LAB07 NO10
.MODEL SMALL     ;Simpan data terkecil di:Reg CL:DX
.STACK 100H
.DATA
DAT DB 65H,8CH,36H   ; 368C65H
    DB 0FFH,45H,77H  ; 7745FFH
    DB 54H,76H,98H   ; 987654H
    DB 0D4H,6CH,0A5H; A56CD4H
    DB 88H,77H,66H   ; 667788H
    DB 66H,45H,77H   ; 774566H
    DB 6FH,8CH,36H   ; 368C6FH
    DB 21H,34H,05H   ; 053421H
.CODE
MAIN    PROC
        MOV AX, @DATA
        MOV DS, AX
        MOV SI, 0
        MOV DL, [DAT+SI]   ; Memuat byte pertama dari data ke dalam register DL
        MOV DH, [DAT+SI+1] ; Memuat byte kedua dari data ke dalam register DH
        MOV CL, [DAT+SI+2] ; Memuat byte ketiga dari data ke dalam register CL
        ADD SI, 3          ; Menambahkan 3 ke SI untuk menggeser ke data berikutnya dalam array DAT

LUP1:                      ; Label untuk awal loop
        CMP CL, [DAT+SI+2] ; Membandingkan byte ketiga dari data saat ini dengan byte ketiga dari data selanjutnya
        JG BSR             ; Melompat ke BSR jika byte ketiga dari data saat ini lebih besar dari byte ketiga dari data selanjutnya
        JL KCL             ; Melompat ke KCL jika byte ketiga dari data saat ini lebih kecil dari byte ketiga dari data selanjutnya
        CMP DH, [DAT+SI+1] ; Membandingkan byte kedua dari data saat ini dengan byte kedua dari data selanjutnya
        JG BSR             ; Melompat ke BSR jika byte kedua dari data saat ini lebih besar dari byte kedua dari data selanjutnya
        JL KCL             ; Melompat ke KCL jika byte kedua dari data saat ini lebih kecil dari byte kedua dari data selanjutnya
        CMP DL, [DAT+SI]   ; Membandingkan byte pertama dari data saat ini dengan byte pertama dari data selanjutnya
        JG BSR             ; Melompat ke BSR jika byte pertama dari data saat ini lebih besar dari byte pertama dari data selanjutn
        JL KCL             ; Melompat ke KCL jika byte pertama dari data saat ini lebih kecil dari byte pertama dari data selanjutn
        JMP KCL            ; Melompat ke KCL jika semua byte sama

BSR:    MOV DL, [DAT+SI]   ; Menyalin byte pertama dari data saat ini ke dalam register DL
        MOV DH, [DAT+SI+1] ; Menyalin byte kedua dari data saat ini ke dalam register DH
        MOV CL, [DAT+SI+2] ; Menyalin byte ketiga dari data saat ini ke dalam register CL

KCL:    ADD SI, 3          ; Menambahkan 3 ke SI untuk menggeser ke data berikutnya dalam array DAT
        CMP SI, 24         ; Membandingkan SI dengan 24 (ukuran total data dalam bit)
        JNE LUP1           ; Melompat kembali ke LUP1 jika SI belum mencapai 24
        MOV AH, 4CH        ; Menetapkan fungsi exit (4CH) ke dalam register AH
        INT 21H            ; Memanggil sistem operasi DOS untuk keluar

MAIN ENDP
END MAIN
```

registers

	H	L
AX	4C	20
BX	00	00
CX	01	98
DX	76	54

Analisa:

Program ini berfungsi untuk mencari data biner bertanda 24 bit dengan nilai terkecil kemudian disimpan pada register CL:DX. Program memiliki cara yang sama dengan nomor sebelumnya. Perbedaannya dalam program yaitu penggunaan JB diganti dengan JL serta JA diganti dengan JG sehingga program akan membandingkan data sebagai data biner bertanda.

F. KESIMPULAN

Praktikum dengan judul “Operasi Blok Memori” menghasilkan kesimpulan sebagai berikut :

1. Pada aplikasi EMU 8086 ini penulis dapat melakukan operasi blok memori. Operasi blok memori itu sendiri berfungsi untuk penyalinan isi suatu memori ke memori lain, mengisi suatu memori yang kosong, dan juga dapat melakukan pertukaran isi antara beberapa memori yang ada. Terdapat banyak instruksi perpindahan string yang dapat digunakan untuk memindahkan data (Transfer Data).
2. Untuk melakukan pemindahan data, data diletakkan terlebih dahulu pada ES:[SI] kemudian akan diterima oleh ES:[DI] lalu disimpan pada blok memori yang dituju.
3. Penyalinan data pada emu8086 dapat dilakukan dengan menyalin secara manual menggunakan loop manual atau menggunakan LEA dan MOVSB, sedangkan Pengisian data pada emu8086 dapat dilakukan dengan cara menggunakan loop dengan alamat memori yang naik atau inkremen serta data yang juga diperbarui setiap loopnya.
4. Pembandingan data dapat dilakukan pada bilangan biner bertanda dan tidak bertanda dengan disesuaikan kode yang digunakannya, seperti contoh untuk bilangan bertanda menggunakan JG untuk lompat jika lebih besar, sedangakan untuk bilangan tidak bertanda menggunakan JA.
5. Pencarian data terkecil dapat dilakukan dengan menyimpan data pada suatu register dan membandingkannya dengan data lainnya satu persatu. Jika data pembanding lebih kecil maka data tersebut dipindahkan ke register tujuan, jika tidak maka akan lompat ke program selanjutnya.

G. SARAN

Saran saya dalam melakukan praktikum ini yaitu ketelitian saat sedang menuliskan program harus ditingkatkan, karena tidak menutup kemungkinan terdapat kesalahan jika penulisan program kurang tepat.

DAFTAR PUSTAKA

Kho, D. “Pengertian Mikroprosesor dan Cara Kerja Mikroprosesor”. (Diakses pada 18 April 2024). Available : https://teknikelektronika.com/pengertian-mikroprosesor-microprocessor-cara-kerjamikroprosesor-adalah/

Catur. 2018. “Pengertian Register dan Jenis-Jenis Register”. (Diakses pada 18 April 2024). Available : http://garudacyber.co.id/artikel/1486-pengertian-register-dan-jenis-jenis-register.

Hasanah, Rahmawati. 2019. Mikroprosesor dan Interface. Bandung: Politeknik Negeri Bandung.